

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 2**

**COURSE NAME:** Artificial Intelligence

**COURSE CODE:** CSA17

---

### **COURSE OUTCOME COVERED**

**CO1:** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

*Assessment Tool Weightage: 50%*

---

**QUESTIONS: 3**

**Duration : 1 Hour**  
**Total Marks:30**

Annexure A

### **Constraint-Based Problem Solving**

---

***Code of Conduct:***

*I Jeshyapriya.R(192524453) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

*Jeshyapriya.R(192524453)*

## Annexure A

### Constraint-Based Problem Solving

#### Q1: Hospital Doctor Shift Scheduling Using Backtracking Search

##### 1. Problem Statement

A hospital needs to assign three doctors (D1, D2, D3) to three shifts: Morning, Afternoon, and Night. Each doctor must be assigned exactly one shift while satisfying all the given constraints.

##### 2. Given Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3 (Morning < Afternoon < Night).
3. Only one doctor can work each shift.
4. D3 cannot work the Morning shift.
5. All doctors must be assigned exactly one shift.

##### 3. Solution Using Backtracking Search

**Step 1:** Assign D1 to the Morning shift.

- D1 = Morning
- Remaining shifts: Afternoon, Night
- D1 cannot work Night, so this assignment is valid.

**Step 2:** Assign D2 to the Afternoon shift.

- D2 = Afternoon
- D2 works before D3, satisfying the ordering constraint.

**Step 3:** Assign D3 to the Night shift.

- D3 = Night
- D3 does not work Morning, so the constraint is satisfied.

##### 4. Constraint Checking

| Constraint                  | Assignment                 | Status    |
|-----------------------------|----------------------------|-----------|
| D1 cannot work Night        | D1 = Morning               | Satisfied |
| D2 must work before D3      | D2 = Afternoon, D3 = Night | Satisfied |
| One doctor per shift        | Each shift has one doctor  | Satisfied |
| D3 cannot work Morning      | D3 = Night                 | Satisfied |
| Every doctor gets one shift | All three doctors assigned | Satisfied |

## 5. Final Valid Schedule

| Shift     | Doctor |
|-----------|--------|
| Morning   | D1     |
| Afternoon | D2     |
| Night     | D3     |

**Conclusion:** Backtracking Search finds a valid schedule by assigning doctors one at a time and checking the constraints at every step. If a constraint is violated, the algorithm goes back and tries another assignment.

## Q2: Robot Pathfinding Using BFS Search Algorithm

### 1. Problem Statement

A robot operates in a  $5 \times 5$  grid. It must move from the Start (S) to the Goal (G) while avoiding obstacles. The robot can move Up, Down, Left, and Right. Each movement has a cost of 1.

The objective is to find the shortest path using Breadth-First Search (BFS).

### 2. Given Constraints

1. The robot can move Up, Down, Left, and Right.
2. Each movement costs 1.
3. The robot cannot pass through obstacles.
4. Manhattan Distance is used as a heuristic.

### 3. Example Grid

Assume the following grid, since the question does not specify the obstacle positions:

```
S . . # .  
. # . # .  
. # . . .  
. . . # .  
# # . . G
```

#### Legend:

- S = Start
- G = Goal
- . = Free cell

- = Obstacle

Start position: (0, 0)

Goal position: (4, 4)

#### 4. Manhattan Distance Heuristic

The Manhattan Distance is calculated using:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

For the Start position (0, 0) and Goal position (4, 4):

$$h(S) = |4 - 0| + |4 - 0|$$

$h(S) = 8$

The heuristic estimates the minimum number of horizontal and vertical moves required to reach the goal, ignoring obstacles.

#### 5. BFS Search Algorithm

**Step 1:** Initialize the queue with the Start node (0, 0).

**Step 2:** Remove the first node from the queue and examine its neighboring cells.

**Step 3:** Ignore cells that are outside the grid, blocked by obstacles, or already visited.

**Step 4:** Add valid neighboring cells to the queue and record their parent nodes.

**Step 5:** Repeat until the Goal node is reached.

**Step 6:** Trace the parent nodes backward to reconstruct the path.

#### 6. Priority Queue / Node Expansion

BFS uses a **FIFO queue**, not a priority queue. It expands nodes in the order they are discovered.

For the example grid, one possible sequence of expansions begins as follows:

| Step | Expanded Node | Action                |
|------|---------------|-----------------------|
| 1    | (0, 0)        | Start                 |
| 2    | (0, 1)        | Expand valid neighbor |
| 3    | (1, 0)        | Expand valid neighbor |
| 4    | (0, 2)        | Continue searching    |

| Step | Expanded Node         | Action                           |
|------|-----------------------|----------------------------------|
| 5    | (2, 0)                | Continue searching               |
| 6    | Other reachable cells | Continue until the goal is found |

The exact expansion order depends on the order in which neighbors are checked.

## 7. BFS vs. Heuristic Search

BFS guarantees the shortest path when all movement costs are equal.

Manhattan Distance is a heuristic commonly used by A\* search:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$  = Actual cost from Start to the current node.
- $h(n)$  = Estimated cost from the current node to Goal.
- $f(n)$  = Estimated total path cost.

BFS uses a FIFO queue, whereas A\* uses a priority queue ordered by  $f(n)$ .

## 8. Final Result

For the example grid, a shortest valid path is:

$(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2)$

$\rightarrow (2,2) \rightarrow (2,3) \rightarrow (2,4) \rightarrow (3,4) \rightarrow (4,4)$

**Total cost = 8 moves.**

**Conclusion:** BFS finds the shortest path in an unweighted grid by exploring nodes level by level. Manhattan Distance can help guide the search when using A\*.

# Q3: Autonomous Rescue Robot Using Search Strategies

## (a) Analyze the Constraints and Explain Their Effect on Decision-Making

### 1. Problem Statement

An autonomous rescue robot operates in a disaster-affected building. It starts at position S and must reach a survivor at position G.

The building contains obstacles, and the robot has limited sensing ability. It must find a safe path while minimizing the total movement cost.

## **2. Given Constraints**

### **1. Movement Constraints**

- The robot can move Up, Down, Left, and Right.
- It cannot move diagonally.
- This limits the available paths.

### **2. Obstacles**

- Some cells are blocked.
- The robot must avoid blocked cells.
- This prevents unsafe or impossible movements.

### **3. Limited Sensing**

- The robot can observe only adjacent cells.
- It cannot know the entire environment in advance.
- It must explore the surroundings before selecting the next move.

### **4. Movement Costs**

- Normal movement costs 1.
- Entering an unknown or risky zone costs an additional 2.

Therefore, if entering a risky zone adds 2 to the normal movement cost:

$$\text{Risky movement cost} = 1 + 2 = 3$$

### **5. Minimum-Cost Requirement**

- The robot must reach the survivor at minimum total cost.
- It should prefer safer and less expensive routes.

### **6. Dynamic Environment**

- New obstacles or hazards may appear.
- The robot must update its knowledge and re-plan its route.

### **3. Effect on Decision-Making**

These constraints require the robot to evaluate possible actions before moving. It must consider the distance to the goal, movement costs, obstacles, and newly discovered information.

The robot should not simply choose the shortest route. It must select a safe route with the lowest total cost.

## **(b) Develop a Solution Approach Using Uniform Cost Search (UCS)**

### **1. Why Uniform Cost Search?**

**Uniform Cost Search (UCS)** is suitable because different movements may have different costs.

UCS expands the node with the lowest accumulated path cost.

$$f(n) = g(n)$$

Where:

- $g(n)$  = Total cost from the Start to the current node.

### **2. Steps Involved**

#### **Step 1: Initialize**

- Place the Start node S in a priority queue.
- Set the initial cost to 0.

#### **Step 2: Observe the Environment**

- Identify the adjacent cells.
- Determine which cells are blocked, safe, or unknown.

#### **Step 3: Expand the Lowest-Cost Node**

- Remove the node with the smallest accumulated cost.
- Examine its valid neighboring cells.

#### **Step 4: Calculate Movement Costs**

- Normal movement = 1.
- Movement into an unknown or risky zone = 3, assuming the additional cost of 2 is added to the normal cost.

Calculate the new path cost:

$$\text{New Cost} = \text{Current Cost} + \text{Movement Cost}$$

#### **Step 5: Update the Priority Queue**

- Add newly discovered valid nodes.
- If a lower-cost path to a node is found, update its cost and parent.

#### **Step 6: Reach the Goal**

- Continue expanding the lowest-cost node.
- Stop when the goal is removed from the priority queue with its minimum cost confirmed.

### Step 7: Re-plan When Necessary

- If a new obstacle or hazard is discovered, update the environment model.
- Recalculate the route and continue toward the survivor.

### 3. Algorithm

1. Start from S with cost 0.
2. Insert S into the priority queue.
3. While the queue is not empty:
  - a. Remove the node with the lowest cost.
  - b. If the node is G, return the path and cost.
  - c. Observe adjacent cells.
  - d. Ignore blocked cells.
  - e. Calculate the cost of each valid movement.
  - f. Update lower-cost paths in the queue.
4. If G cannot be reached, report failure.

### 4. Example

Assume two possible routes exist:

| Route   | Movement | Cost | Risk | Cost | Total Cost |
|---------|----------|------|------|------|------------|
| Route A | 6        |      | 4    |      | 10         |
| Route B | 8        |      | 0    |      | 8          |

Route A is shorter in terms of the number of movements, but it contains risky zones.

Route B has more movements but a lower total cost.

Therefore, UCS selects **Route B** because its total cost is 8, which is lower than Route A's cost of 10.

## **(c) Demonstrate How the Robot Applies AI Concepts**

### **1. Intelligent Agent**

An intelligent agent observes its environment, makes decisions, and performs actions to achieve a goal.

#### **Application:**

- The robot observes adjacent cells.
- It identifies obstacles and risky zones.
- It selects a suitable path.
- It moves toward the survivor.

Thus, the robot acts as an intelligent agent.

### **2. Rationality**

A rational agent selects actions that maximize its expected performance based on available information.

#### **Application:**

- Avoids known obstacles.
- Considers movement and risk costs.
- Selects the lowest-cost safe route based on current knowledge.
- Changes its decision when new hazards are discovered.

The robot is rational because it chooses actions that support safe and efficient rescue.

### **3. Environment Type**

The robot operates in an environment with the following characteristics:

| <b>Environment Type</b>     | <b>Classification</b>                          | <b>Reason</b>                                                 |
|-----------------------------|------------------------------------------------|---------------------------------------------------------------|
| Observability               | Partially observable                           | The robot observes only adjacent cells.                       |
| Determinism                 | Partially predictable / potentially stochastic | Outcomes may depend on changing hazards.                      |
| Episodic or Sequential      | Sequential                                     | Each movement affects future decisions.                       |
| Static or Dynamic           | Dynamic                                        | Obstacles and hazards may change.                             |
| Discrete or Continuous      | Discrete                                       | The building is represented as grid cells.                    |
| Single-Agent or Multi-Agent | Single-agent model                             | The robot is the decision-making agent in the stated problem. |

### **4. Dynamic Knowledge Updating**

The robot updates its internal map whenever it discovers new information.

For example:

1. The robot moves toward an unknown cell.
2. It detects an obstacle.
3. It marks that cell as blocked.
4. It updates the available routes.
5. It runs the search strategy again to find a safe path.

This process allows the robot to adapt to changing conditions.

#### **(d) Justify Why Uniform Cost Search Is Suitable**

Uniform Cost Search is appropriate for this rescue problem for the following reasons:

##### **1. Handles Different Costs**

Normal and risky movements have different costs. UCS considers the accumulated cost of each path instead of simply counting movements.

##### **2. Finds the Minimum-Cost Path**

When all movement costs are non-negative, UCS finds an optimal path to the goal if one exists and the search is implemented correctly.

##### **3. Avoids Obstacles**

Blocked cells can be excluded from the search, preventing the robot from selecting paths through known obstacles.

##### **4. Supports Re-planning**

When the robot discovers new obstacles or hazards, it can update its map and perform the search again.

##### **5. Suitable for Rescue Operations**

The robot must balance movement distance and risk. UCS compares routes based on total cost and selects the least-cost route according to the defined cost model.

#### **Limitation**

UCS does not use a heuristic to estimate the distance to the goal. It may explore many nodes before finding the destination. Also, the optimality of its route depends on the accuracy of the movement-cost model and the known environment.

## Conclusion

Uniform Cost Search is a suitable strategy for the autonomous rescue robot because it finds the minimum-cost path while considering obstacles and risky zones. Combined with local sensing and dynamic re-planning, it helps the robot adapt to changing conditions and reach the survivor efficiently

## RUBRICS FOR CALCULATION:

| Criteria                | Weightage | Level 4<br>(Exemplary)                                                         | Level 3<br>(Proficient)                               | Level 2<br>(Developing)                       | Level 1<br>(Beginning)                   |
|-------------------------|-----------|--------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------|------------------------------------------|
| Problem Understanding   | 25%       | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps           | Partial understanding; misses key constraints | Fails to identify constraints correctly  |
| Constraint Analysis     | 25%       | Analyzes constraints effectively and prioritizes them logically                | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization         | No meaningful analysis or prioritization |
| Solution Development    | 25%       | Develops optimal and feasible solutions satisfying all constraints             | Develops workable solutions with minor limitations    | Solutions partially satisfy constraints       | Solutions do not meet constraints        |
| Application of Concepts | 15%       | Effectively applies appropriate concepts, methods, or tools                    | Applies concepts with minor errors                    | Limited application; requires guidance        | Unable to apply relevant concepts        |
| Justification           | 10%       | Provides strong justification for decisions with logical reasoning             | Provides justification with some clarity              | Weak or unclear justification                 | No justification provided                |